

AI Usage Note – Pipeline Failure RCA Bot

Overview

Pipeline Failure RCA Bot is an AI-powered Root Cause Analysis system designed to automatically investigate ETL and Cron pipeline failures.

The system analyzes failure logs, compares them with historical successful executions, correlates GitHub code changes, and generates Root Cause Analysis (RCA) reports with recommended fixes and retry procedures.

The platform reduces manual debugging effort and accelerates incident resolution through intelligent automation.

What AI Helped With

Failure Log Analysis

AI was used to analyze uploaded pipeline failure logs and identify:

- Error Messages
- Stack Traces
- Exception Types
- Failed Components
- Execution Context

Instead of manually inspecting large log files, AI quickly extracts relevant failure information.

Root Cause Identification

The AI model identifies probable root causes behind failures.

Examples include:

- Missing Configuration
- Database Connection Failure
- API Timeout
- Null Reference Errors
- KeyError Exceptions
- Dependency Issues

Example:

Failure Log:

KeyError: 'timestamp'

AI Output:

```
{  
  "root_cause": "Payload schema mismatch",  
  "severity": "Critical",  
  "confidence": 0.95  
}
```

Severity Classification:

AI automatically classifies incidents into severity levels.

Possible severity levels:

- Low
- Medium
- High
- Critical

Example:

Error Type	Severity
Warning	Low
API Timeout	Medium
Database Failure	High
Application Crash	Critical

This helps engineering teams prioritize incidents.

Confidence Score Generation

For every RCA result, AI generates a confidence score.

Example:

```
{  
  "severity": "Critical",  
  "confidence": 0.95  
}
```

Higher confidence indicates stronger certainty in the generated analysis.

GitHub Change Correlation

AI reviews recent GitHub commit changes and attempts to identify code modification that may have introduced the failure.

Example:

Commit:

Refactor dictionary response keys

Detected Change:

timestamp -> ts

AI determines whether the change is related to the observed failure.

Recommended Fix Generation

AI generates suggested code fixes.

Example:

Update downstream loader to use 'ts' field or restore backward compatibility with 'timestamp'

This reduces debugging and troubleshooting time.

Retry Recommendation Generation

AI generates recovery steps.

Example:

1. Apply code fix
2. Execute validation tests
3. Deploy changes
4. Re-run pipeline

Technologies Used

Artificial Intelligence

- Ollama
- Mistral LLM
- Prompt Engineering
- Natural Language Processing (NLP)

Backend Technologies

- Python
- Flask
- SQLAlchemy / MongoDB
- REST APIs

Frontend Technologies

- HTML
- CSS
- JavaScript

Integrations

- GitHub API
- Pipeline Log Files
- AI Analysis Engine

What AI Got Wrong (and How We Fixed It)

Incorrect Severity Classification

Some failures were initially classified incorrectly.

Fix:

- Severity scoring rules were introduced.
- Critical keywords receive higher priority.

Inaccurate Root Cause Detection

Logs with insufficient information could lead to weak RCA results.

Fix:

- Historical successful logs were used for comparison.
- Additional context was added to prompts.

Missing GitHub Correlation

When GitHub repositories were not configured, commit analysis was unavailable.

Fix:

- GitHub Token integration was implemented.
- Fallback simulation mode was added

Low Confidence Predictions

Some analyses returned uncertain results.

Fix:

- Confidence scoring was introduced.
- Reports below threshold can be reviewed manually.

Human Contributions

While AI performs log analysis and RCA generation, the overall solution was designed and implemented by the development team.

Human responsibilities included:

- System Architecture Design
- Flask Backend Development
- Database Design
- GitHub API Integration
- Log Processing Logic
- Severity Classification Rules
- UI Development
- Error Handling
- Testing and Validation
- Documentation

AI acted as an intelligent analysis engine while engineering decisions remained under human control.

Key Prompts Used

Root Cause Analysis Prompt

"Analyze the pipeline failure log and identify the most probable root cause."

Severity Classification Prompt

"Determine whether the incident should be classified as Low, Medium, High, or Critical."

GitHub Correlation Prompt

"Determine whether the incident should be classified as Low, Medium, High, or Critical."

Fix Recommendation Prompt

"Generate corrective actions and recommended retry steps for resolving the incident."

Benefits Achieved

Pipeline Failure RCA Bot provides:

- Faster Incident Investigation
- Reduced Manual Debugging
- Automated Root Cause Identification
- Severity Classification
- GitHub Change Correlation
- AI-Generated Recommendations
- Historical RCA Tracking
- PDF Report Generation
- Improved Operational Reliability

Conclusion:

Pipeline Failure RCA Bot demonstrates the practical application of Artificial Intelligence in DevOps and incident management. By combining Python, Flask, GitHub integration, pipeline log analysis, and Large Language Models, the system delivers an intelligent and scalable solution for automated Root Cause Analysis.

AI significantly reduces troubleshooting effort, accelerates incident resolution, and helps engineering teams maintain reliable production systems.

